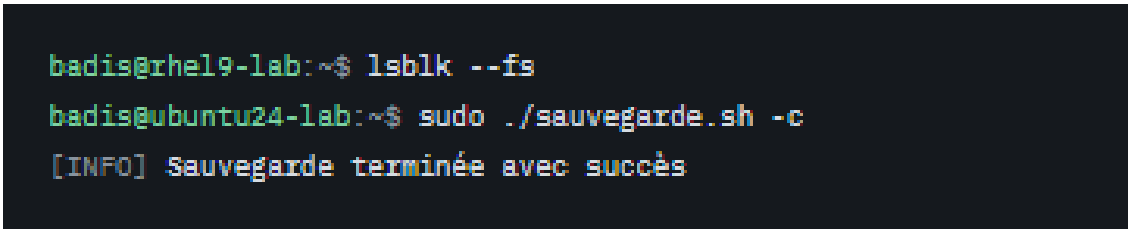
Travaux pratiques · Administration système

Administration Linux

Installation, partitionnement LVM, sécurisation SSH et automatisation par scripts Bash



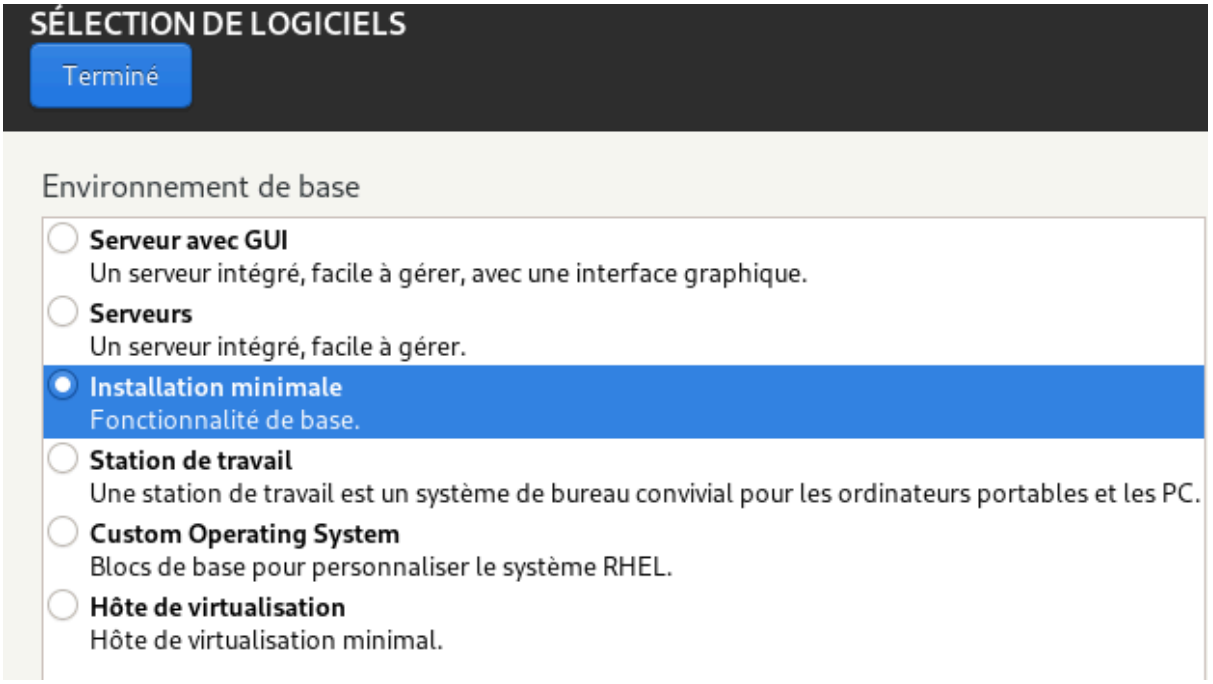
Réalisé par
Badis WACHANI
Geofrey FRAY

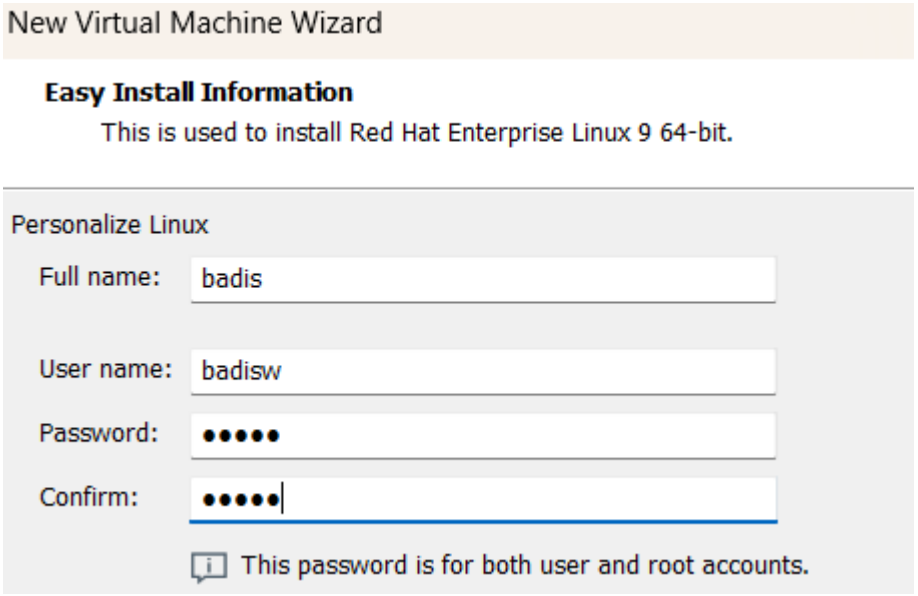
Encadré par
Sofiane BOULJARF

TP 1.1 : Installer et partitionner deux serveurs (VM Red Hat)

Installation des deux machines partitionnées et à jour :
rhel9-lab

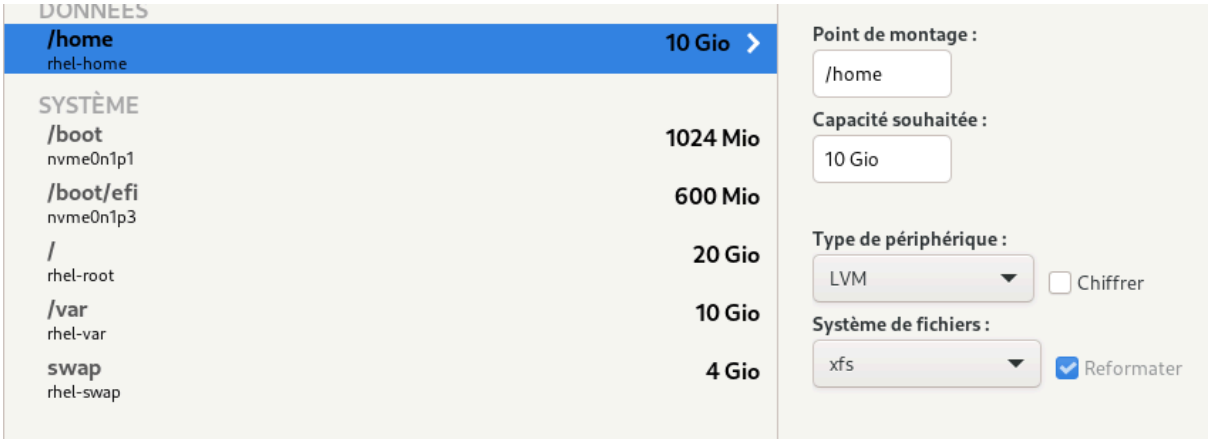
VM pour la redhat :

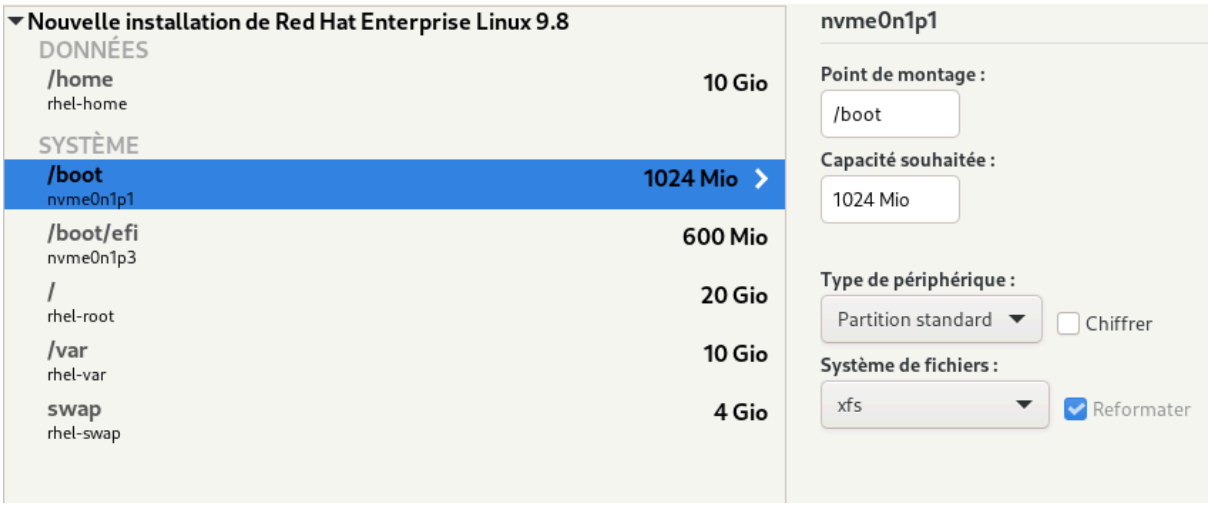


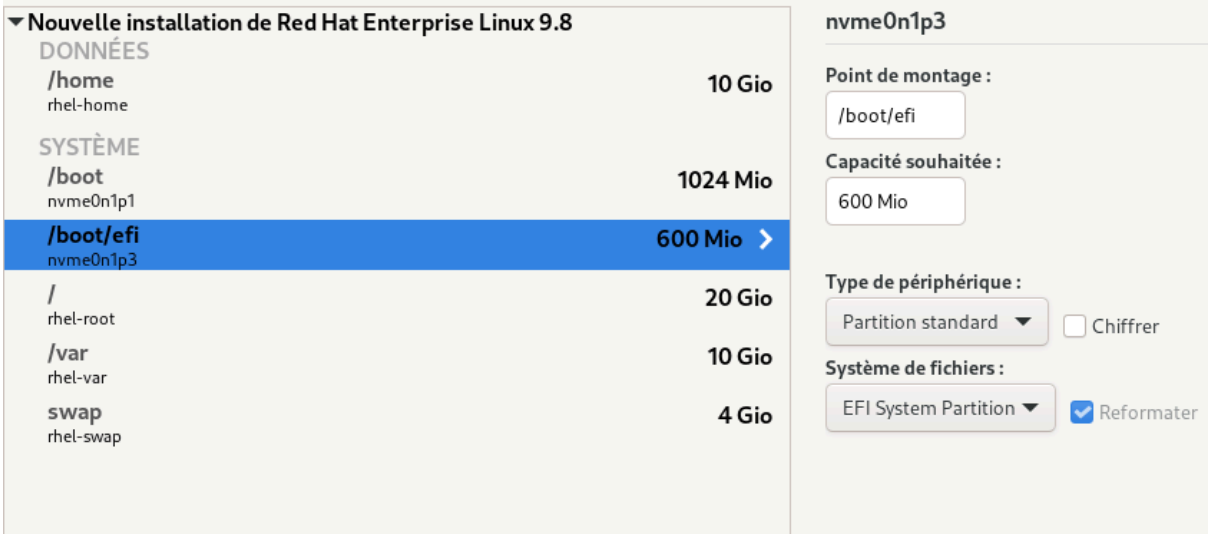


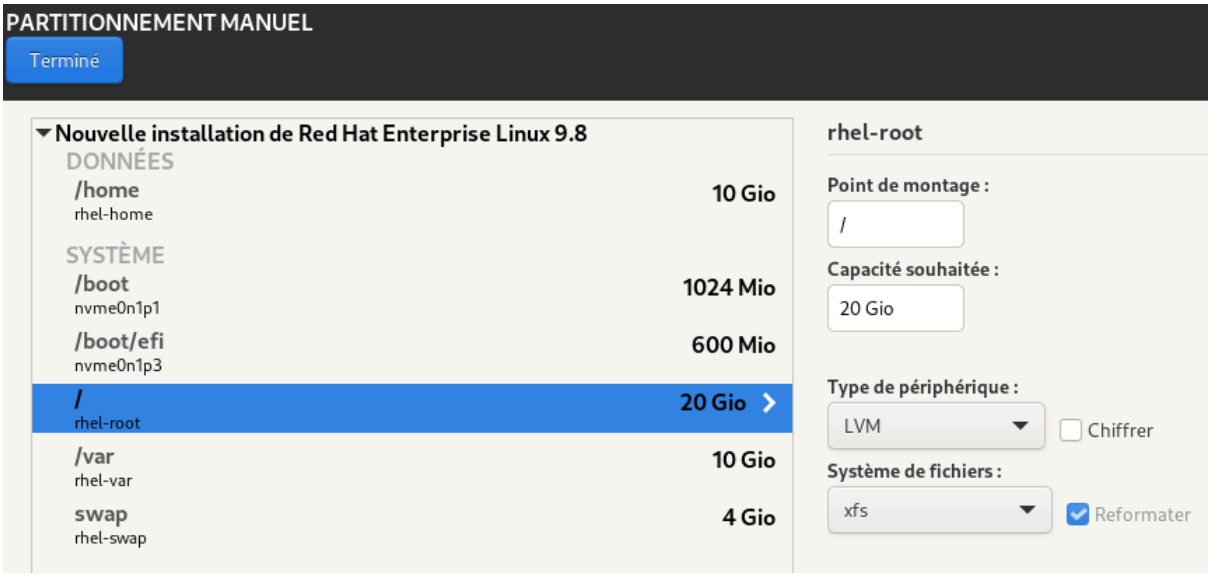
Partition : VM RED HAT
60 gio en UEFI

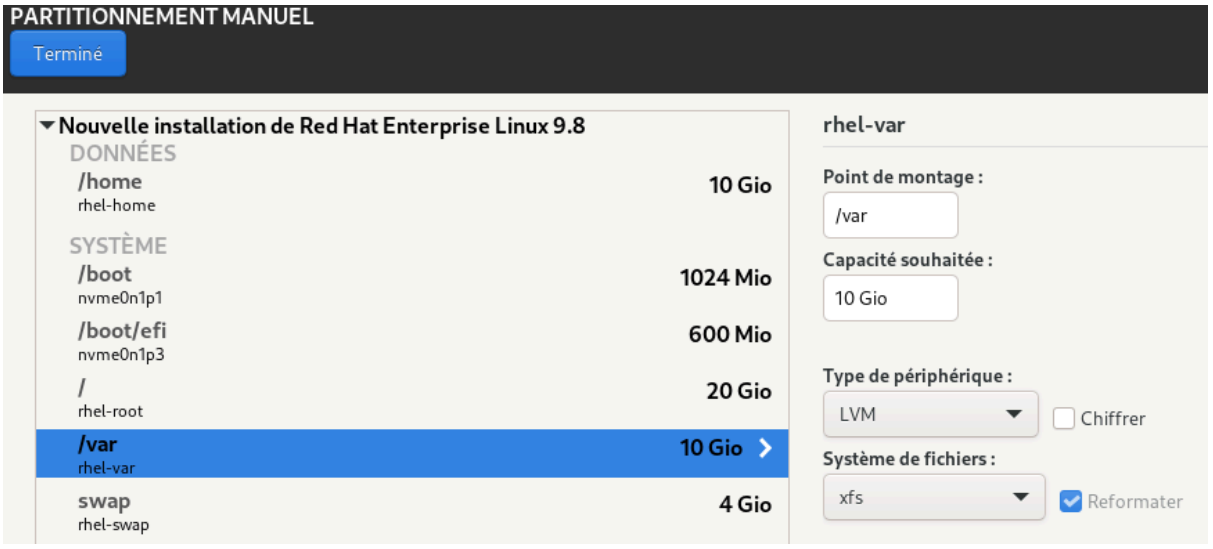
LVM : On ajoute une **couche intermédiaire**. Le disque est d'abord versé dans un réservoir, puis on puise dans ce réservoir pour créer des volumes.
Partition standard : Le disque est découpé **directement** en morceaux, posés les uns à côté des autres, comme des tranches de gâteau.

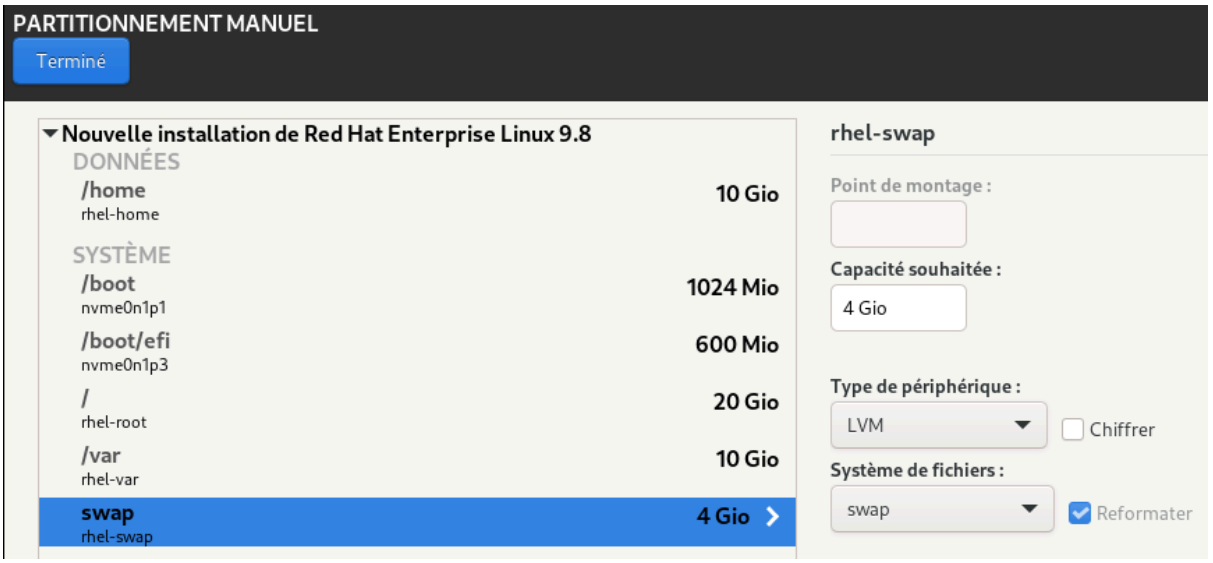


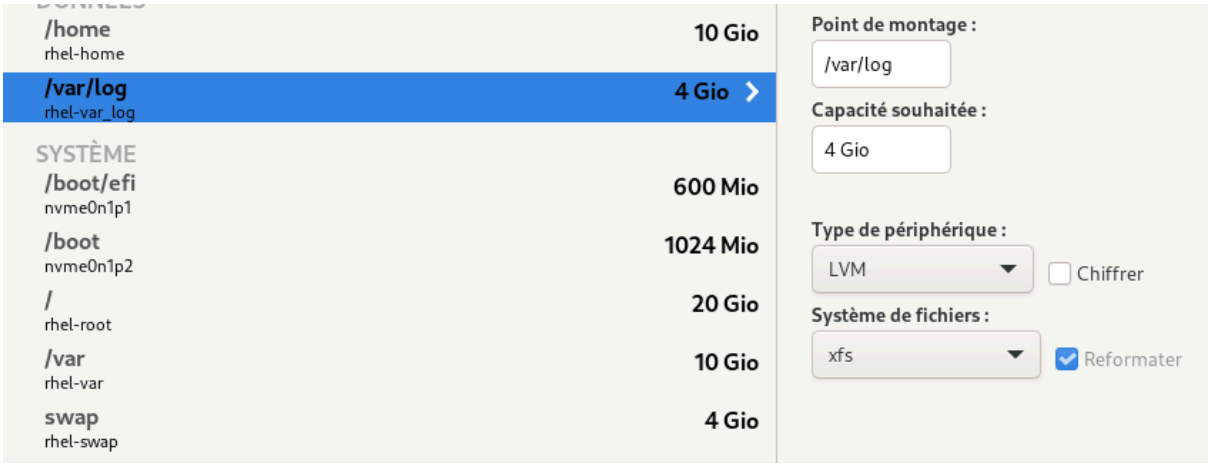





PARTITIONNEMENT MANUEL
Terminé
Nouvelle installation de Red Hat Enterprise Linux 9.8
DONNÉES
/home
rhel-home
10 Gio
SYSTÈME
/boot
nvme0n1p1
1024 Mio
/boot/efi
nvme0n1p3
600 Mio
/
rhel-root
20 Gio
/var
rhel-var
10 Gio
swap
rhel-swap
4 Gio
rhel-root
Point de montage :
/
Capacité souhaitée :
20 Gio
Type de périphérique :
LVM
Chiffrer
Système de fichiers :
xfs
Reformater

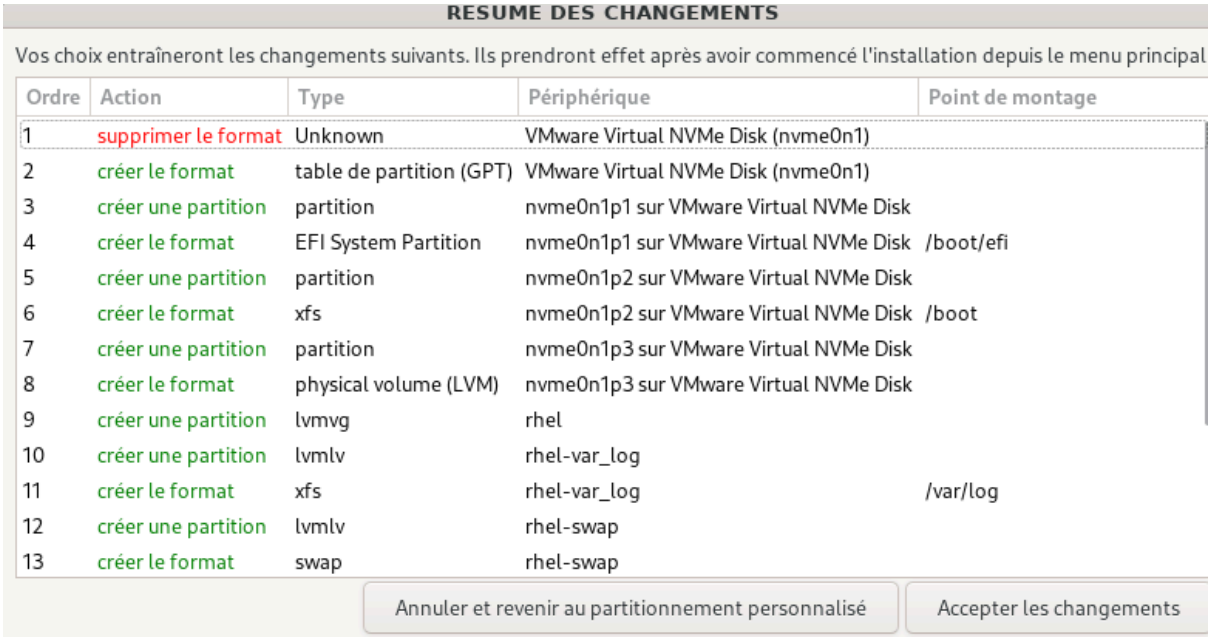

PARTITIONNEMENT MANUEL
Terminé
Nouvelle installation de Red Hat Enterprise Linux 9.8
DONNÉES
/home
rhel-home
10 Gio
SYSTÈME
/boot
nvme0n1p1
1024 Mio
/boot/efi
nvme0n1p3
600 Mio
/
rhel-root
20 Gio
/var
rhel-var
10 Gio
swap
rhel-swap
4 Gio
rhel-var
Point de montage :
/var
Capacité souhaitée :
10 Gio
Type de périphérique :
LVM
Chiffrer
Système de fichiers :
xfs
Reformater


PARTITIONNEMENT MANUEL
Terminé
Nouvelle installation de Red Hat Enterprise Linux 9.8
DONNÉES
/home
rhel-home
10 Gio
SYSTÈME
/boot
nvme0n1p1
1024 Mio
/boot/efi
nvme0n1p3
600 Mio
/
rhel-root
20 Gio
/var
rhel-var
10 Gio
swap
rhel-swap
4 Gio
rhel-swap
Point de montage :
Capacité souhaitée :
4 Gio
Type de périphérique :
LVM
Chiffrer
Système de fichiers :
swap
Reformater



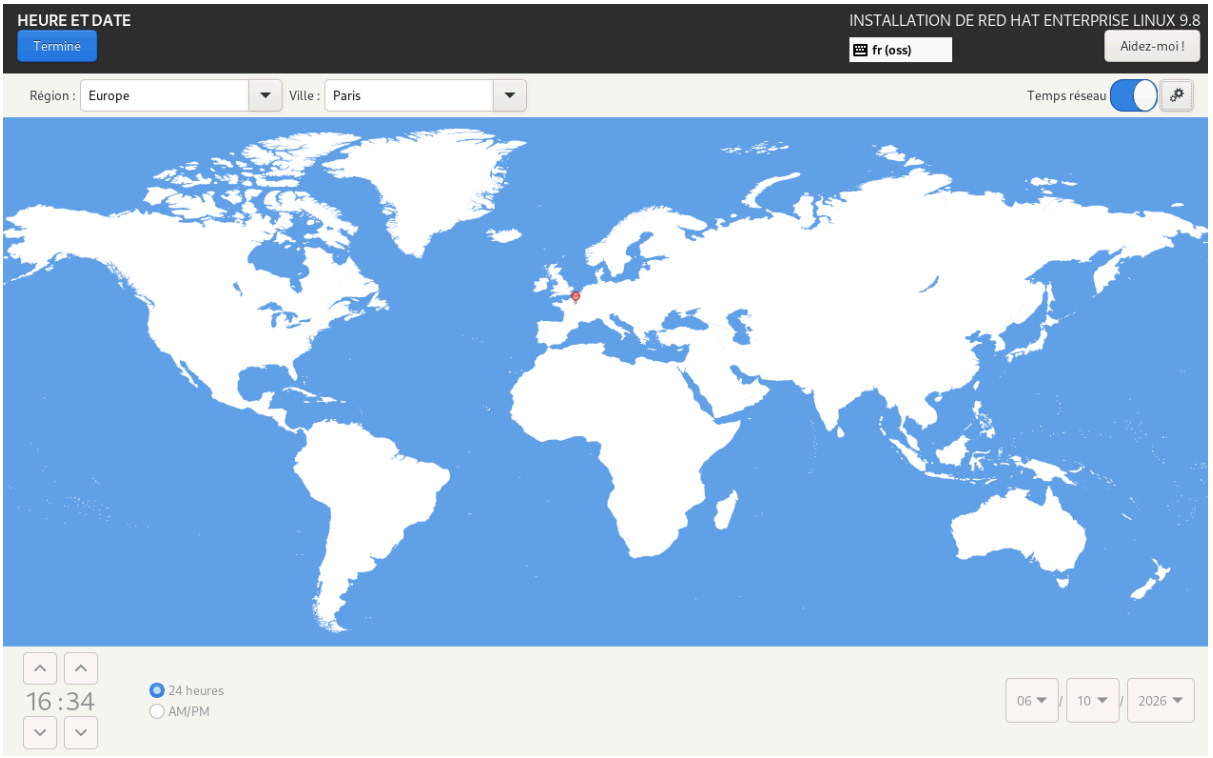
On a choisi XFS car il gère des systèmes de fichiers et des fichiers beaucoup plus grands qu'ext4. XFS découpe le disque en zones indépendantes. Combiné a LVM, on agrandit un volume XFS sans démonter ni arrêter quoi que ce soit.

4.



14	créer une partition	lvmlv	rhel-var	
15	créer le format	xfs	rhel-var	/var
16	créer une partition	lvmlv	rhel-root	
17	créer le format	xfs	rhel-root	/
18	créer une partition	lvmlv	rhel-home	
19	créer le format	xfs	rhel-home	/home

5.



6.1

Identité : Donne le nom rhel9-lab à la machine et vérifié via hostnamectl

```
[badis@localhost ~]$ sudo hostnamectl set-hostname rhel9-lab
[sudo] Mot de passe de badis :
[badis@localhost ~]$ hostnamectl
 Static hostname: rhel9-lab
       Icon name: computer-vm
```

6.2 : Fuseau Horaire / vérifier avec timedatectl

```
[badis@localhost ~]$ sudo timedatectl set-timezone Europe/Paris
[badis@localhost ~]$ timedatectl
               Local time: mer. 2026-10-07 09:57:23 CEST
           Universal time: mer. 2026-10-07 07:57:23 UTC
                 RTC time: mer. 2026-10-07 07:57:23
                Time zone: Europe/Paris (CEST, +0200)
System clock synchronized: yes
```

6.3 : Enregistrer la machine chez Red Hat

```
[badis@localhost ~]$ sudo subscription-manager register --username badis.wachani@laplateforme.io
Inscription sur : subscription.rhsm.redhat.com:443/subscription
Mot de passe :
Le système a été enregistré avec l'ID : 60cad445-8ded-4ee9-ae39-648fb9173e23
Le nom de système enregistré est : rhel9-lab
```

6.4

```
[root@rhel9-lab badis]# sudo passwd -l root
Verrouillage du mot de passe pour l'utilisateur root.
passwd: Succès
[root@rhel9-lab badis]# sudo passwd -S root
root LK 1969-12-31 0 99999 7 -1 (Mot de passe verrouillé.)
[root@rhel9-lab badis]# sudo sshd -T | grep -i permitrootlogin
permitrootlogin yes
```

On a verrouillé le mot de passe de root et vérifie que root ne peut pas se connecter en SSH
Puis on va faire en sorte qu'on puisse se connecter qu'en sudo.

```
[root@rhel9-lab badis]# echo "PermitRootLogin no" | sudo tee /etc/ssh/sshd_config.d/01-no-root.conf
sudo systemctl restart sshd
PermitRootLogin no
```

6.5

```
[root@rhel9-lab badis]# sudo systemctl enable --now firewalld
[root@rhel9-lab badis]# sudo firewall-cmd --state
running
[root@rhel9-lab badis]# sudo firewall-cmd --list-all
public (active)
  target: default
  icmp-block-inversion: no
  interfaces: ens160
  sources:
  services: cockpit dhcpv6-client ssh
```

On a activé le pare-feu (a chaque démarrage constant),puis on a vérifié si le service est en route puis on a affiché ce qui est autorisé à entrer.

6.6
Lister les services qui ont échoué au démarrage :

```
[root@rhel9-lab badis]# systemctl --failed
  UNIT LOAD ACTIVE SUB DESCRIPTION
0 loaded units listed.
```

6.7

```
[root@rhel9-lab badis]# sudo subscription-manager refresh
sudo dnf repolist
Toutes les données locales réactualisées
Mise à jour des référentiels de gestion des abonnements.
id du dépôt                                  nom du dépôt
rhel-9-for-x86_64-appstream-rpms             Red Hat Enterprise Linux 9 for x86_64 - AppStream (RPMs)
rhel-9-for-x86_64-baseos-rpms                Red Hat Enterprise Linux 9 for x86_64 - BaseOS (RPMs)
```

On a enfin récupéré les droits depuis Red Hat et liste les dépôts.
après (2 jours de combat.)

6.8

```
[root@rhel9-lab badis]# sudo dnf upgrade -y
Mise à jour des référentiels de gestion des abonnements.
Dernière vérification de l'expiration des métadonnées effectuée il y a 0:21:46 le jeu. 08 oct. 2026 09:35:43.
Dépendances résolues.
================================================================================
 Paquet                  Architecture
                                 Version                        Dépôt                    Taille
================================================================================
Installation:
 kernel                  x86_64  5.14.0-687.54.1.el9_8          rhel-9-for-x86_64-baseos-rpms  1.0 M
Mise à jour:
```

On a pour 132 paquets de mise à jour.

```
Exécution du scriptlet: microcode_ctl-4:20260210-1.20260812.1.el9_8.noarch
                                                                  105/260
Exécution du scriptlet: microcode_ctl-4:20260210-1.20260812.1.el9_8.noarch  105/260
Exécution du scriptlet: openssh-server-9.9p1-12.el9_8.x86_64              106/260
Mise à jour de        : openssh-server-9.9p1-12.el9_8.x86_64              106/260
Exécution du scriptlet: openssh-server-9.9p1-12.el9_8.x86_64              106/260
Mise à jour de        : mdadm-4.4-4.el9_8.2.x86_64                        107/260
Exécution du scriptlet: mdadm-4.4-4.el9_8.2.x86_64                        107/260
Mise à jour de        : sudo-1.9.17p2-3.el9_8.3.x86_64                    108/260
Exécution du scriptlet: sudo-1.9.17p2-3.el9_8.3.x86_64                    108/260
Mise à jour de        : openssh-clients-9.9p1-12.el9_8.x86_64             109/260
```

6.9

pour le SSH :

```
C:\Users\badis>ssh badis@192.168.184.153
badis@192.168.184.153's password:
```

terminal powershell → vm

6.10

```
[root@rhel9-lab badis]# sudo dnf install -y glibc-langpack-fr
Mise à jour des référentiels de gestion des abonnements.
Dernière vérification de l'expiration des métadonnées effectuée il y a 0:43:05 le jeu. 08 oct. 2026 09:35:43.
sLe paquet glibc-langpack-fr-2.34-275.el9_8.x86_64 est déjà installé.
Dépendances résolues.
Rien à faire.
Terminé !
```

et pour vérifier tu fais juste la commande localectl

```
[root@rhel9-lab badis]# localectl
System Locale: LANG=fr_FR.UTF-8
    VC Keymap: fr
   X11 Layout: fr
    X11 Model: pc105
  X11 Options: terminate:ctrl_alt_bksp
```

7.1 : l'arborescence des disques : lsblk -f

Elle montre le disque, ses partitions et les volumes LVM, avec leur format (xfs, swap)
et leur point de montage. juste pour vérifier si notre schéma de partitionnement a bien été appliqué.

```
[badis@rhel9-lab ~]$ lsblk -f
NAME          FSTYPE  FSVER   LABEL                  UUID                                   FSAVAIL FSUSE% MOUNTPOINTS
sr0           iso9660         CDROM                  2026-10-06-21-25-23-00
sr1           iso9660 Joliet  RHEL-9-8-0-BaseOS-x86_64 2026-05-11-17-14-45-00
nvme0n1
├─nvme0n1p1   vfat    FAT32                          FB4D-26F7                               591,2M     1% /boot/efi
├─nvme0n1p2   xfs                                    6dfe087f-720d-4bf7-bf03-6f2df1dec240     473M    51% /boot
├─nvme0n1p3   LVM2_me LVM2 00                        9Uqr4w-y1qJ-3wA8-2ULv-i9KX-M0rh-rv1Tx2
│ ├─rhel-root xfs                                    2a4e922c-7f82-468e-8925-368f9ea6c91e     18,1G     9% /
│ ├─rhel-swap swap    1                              ab7fcc0e-2604-4254-8dd6-6c050f3f3676                 [SWAP]
│ ├─rhel-home xfs                                    ab302f35-8fa2-4f25-8ac9-af9a162d6c0f      9,8G     1% /home
│ └─rhel-var  xfs                                    e3e3ae99-eec8-4b74-9732-31de191a3365      9,3G     6% /var
└─nvme0n1p4   xfs                                    54b8c3c0-54e2-4d4d-9c27-cddfcb31dce1      3,9G     2% /var/log
```

l’espace utilisé : df -hT
Montre sa taille, l’espace utilisé, et chaque système de fichiers monté et son type

```
[badis@rhel9-lab ~]$ df -hT
Sys. de fichiers      Type     Taille Utilisé Dispo Uti% Monté sur
devtmpfs              devtmpfs   1,8G       0  1,8G   0% /dev
tmpfs                 tmpfs      1,8G       0  1,8G   0% /dev/shm
tmpfs                 tmpfs      725M     11M  715M   2% /run
efivarfs              efivarfs   256K     57K  195K  23% /sys/firmware/efi/efivars
/dev/mapper/rhel-root xfs         20G    1,9G   19G  10% /
/dev/nvme0n1p2        xfs        960M    488M  473M  51% /boot
/dev/nvme0n1p1        vfat       599M    7,7M  592M   2% /boot/efi
/dev/mapper/rhel-var  xfs         10G    634M  9,4G   7% /var
/dev/mapper/rhel-home xfs         10G    104M  9,9G   2% /home
/dev/nvme0n1p4        xfs        4,0G     72M  3,9G   2% /var/log
tmpfs                 tmpfs      363M       0  363M   0% /run/user/1000
```

la mémoire d’échange : swapon –show
les espaces swap actifs et leur taille.
que le swap est bien activé.

```
[badis@rhel9-lab ~]$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/dm-1 partition   4G   0B   -2
```

les services en échec et puis l’historique des opérations dnf (installations, mise à jour) :

```
[badis@rhel9-lab ~]$ systemctl --failed
  UNIT LOAD ACTIVE SUB DESCRIPTION
0 loaded units listed.
[badis@rhel9-lab ~]$ sudo dnf history
Mise à jour des référentiels de gestion des abonnements.
ID     | Ligne de commande                                                                | Date et heure    | Action(s)      | Modifié
-------------------------------------------------------------------------------------------------------------------------------
     2 | upgrade -y                                                                       | 2026-10-08 09:59 | I, U           |  132  <
     1 |                                                                                  | 2026-10-07 09:07 | Install        |  434 >E
[badis@rhel9-lab ~]$
```

Q1.1 — Pourquoi /boot reste-t-il une partition classique plutôt qu'un volume logique ?

Le /boot ne sera jamais rempli, donc on part sur une partition classique , c’est plus simple et plus fiable pour qu’il y accède et en plus /boot a une petite taille et donc il est stable on a pas besoin d'agrandir son stockage.

Q1.2 — Décrivez ce qui se passe si une application remplit /var/log : d'abord quand il est sur la racine, ensuite quand il est séparé.

si il est sur la racine, et que la racine est saturée alors services plantent, impossible mise a jour et même système impossible a redémarré.

Alors que si on le sépare de la racine, on perd les nouveaux journaux, mais le système n’est pas impacté.

Q1.3 — Pourquoi conserver de l'espace libre non alloué plutôt que de tout attribuer ?
Pour que plus tard on puisse agrandir le volume logique dont on a besoin.

TP 1.2 : Réseau fixe et SSH durci par clés

1.2.1 Les interfaces réseau

```
[badis@rhel9-lab ~]$ ip -br link
lo               UNKNOWN        00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
ens160           UP             00:0c:29:22:f0:ff <BROADCAST,MULTICAST,UP,LOWER_UP>
```

Nous liste cartes réseau, et leur état si UP / DOWN puis leur adresse MAC et puis le nom de l'interface. le -br c'est juste pour un affichage court.

1.2.1 Les adresses ip

```
[badis@rhel9-lab ~]$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens160           UP             192.168.184.153/24 fe80::20c:29ff:fe22:f0ff/64
```

Adresse IP de l'interface, avec son masque donc pour nous c'est ens160 vue que c'est en UP. le 24/ signifie = 255.255.255.0 soit 254 machine possible sur le réseau.
et si on veut voir que pour ens160
ip addr show ens160

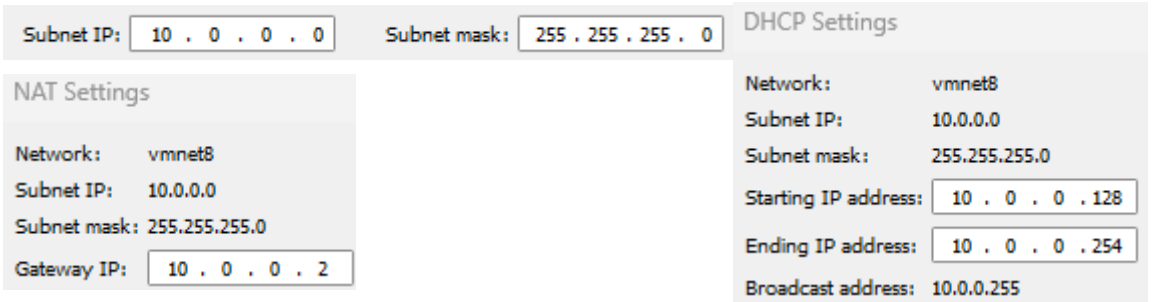
```
[badis@rhel9-lab ~]$ ip addr show ens160
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:22:f0:ff brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 192.168.184.153/24 brd 192.168.184.255 scope global dynamic noprefixroute ens160
       valid_lft 1792sec preferred_lft 1792sec
    inet6 fe80::20c:29ff:fe22:f0ff/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

1.2.1. La route par défaut

```
[badis@rhel9-lab ~]$ ip route
default via 192.168.184.2 dev ens160 proto dhcp src 192.168.184.153 metric 100
192.168.184.0/24 dev ens160 proto kernel scope link src 192.168.184.153 metric 100
```

Elle montre la table de routage.
dev ens160 : interface utilisée
proto dhcp : la route fournie par le DHCP

1.2.2




```
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:22:f0:ff brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 10.0.0.10/24 brd 10.0.0.255 scope global noprefixroute ens160
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe22:f0ff/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

juste pour vérifier : on est en static au lieu de dhcp.

```
[badis@rhel9-lab ~]$ ip route
default via 10.0.0.2 dev ens160 proto static metric 100
10.0.0.0/24 dev ens160 proto kernel scope link src 10.0.0.10 metric 100
```

1.2.3
Générer la paire de clé SSH
la phrase : [redacted]

```
[badis@rhel9-lab ~]$ ssh-keygen -t ed25519 -C "badis@redhatubuntu"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/badis/.ssh/id_ed25519):
Enter passphrase for "/home/badis/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/badis/.ssh/id_ed25519
Your public key has been saved in /home/badis/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:rEMAptdQFv7mylU9AxZEe2IJgbrT2HERfjg1dO/oAJA badis@redhatubuntu
The key's randomart image is:
+--[ED25519 256]--+
|  +.=++*B .      |
| o *Eo.+ * .     |
|. ..+ =.O . .    |
| .. .o.B = o     |
|   = o+ S = .    |
|  + o+ o o o     |
|   .  =   .      |
|   . o .         |
|    o            |
+----[SHA256]-----+
```

Pour vérifier qu'elle sont bien générer :

```
[badis@rhel9-lab ~]$ ls -l ~/.ssh/
total 12
-rw-------. 1 badis badis 101  8 oct.  14:37 authorized_keys
-rw-------. 1 badis badis 464  8 oct.  15:01 id_ed25519
-rw-r--r--. 1 badis badis 100  8 oct.  15:01 id_ed25519.pub
```

```
C:\Users\badis>ssh badis@10.0.0.10
Enter passphrase for key 'C:\Users\badis/.ssh/id_ed25519':
```

Côté serveur, la clé publique va dans ~/.ssh/authorized_keys. Droits obligatoires : 700 sur ~/.ssh, 600 sur authorized_keys (sinon sshd refuse la clé).

1.2.4 : Fichier de configuration
L'emplacement du fichier de configuration client SSH :
-p = crée un répertoire sans erreur s'il existe déja ~ = dossier personnel de l'utilisateur

```
[badis@rhel9-lab ~]$ chmod 700 ~/.ssh/config
[badis@rhel9-lab ~]$ ls -l ~/.ssh/config
-rwx------. 1 badis badis 128  8 oct.  15:09 /home/badis/.ssh/config
```

chmod 700 car badis est le propriétaire.

```
[badis@rhel9-lab ~]$ nano ~/.ssh/config
```

```
Host rhel9-lab rhel
    HostName 10.0.0.10
    User badis
    Port 22
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
```

Port 22 : SSH

Après les changements du coup redéployer la clé sur la VM RHEL9.

```
[badis@rhel9-lab ~]$ ssh-copy-id rhel9-lab
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/badis/.ssh/id_ed25519.pub"
The authenticity of host '10.0.0.10 (10.0.0.10)' can't be established.
ED25519 key fingerprint is SHA256:Kzxxf4BmFiFijNHSI2+nItKbhu9GVozn3XF9uk9U4CY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
badis@10.0.0.10's password:
Permission denied, please try again.
badis@10.0.0.10's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'rhel9-lab'"
and check to make sure that only the key(s) you wanted were added.
```

1.2.5 : les six directives d'applications

PermitRootLogin : no / root ne se connecte pas en SSH

PasswordAuthentication : no / plus de connexion par mdp

PubkeyAuthentication : yes / seule connexion par clé

MaxAuthTries : 3 / coupe au bout de 3 tentative

X11Forwarding : no / Désactivé affichage GUI

AllowUsers : badis / seul badis peut se log en SSH

```
#SSH - RedHat
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
X11Forwarding no
AllowUsers badis
```

```
[badis@rhel9-lab ~]$ sudo nano /etc/ssh/sshd_config
```

sudo sshd -t = c'est juste pour valider la syntaxe et pour vérifier aucune erreur

sudo systemctl reload sshd = recharger le dossier

pour vérifier les directive à la fois :

```
[badis@rhel9-lab ~]$ sudo sshd -T | grep -i permitrootlogin
sudo sshd -T | grep -i passwordauthentication
sudo sshd -T | grep -i pubkeyauthentication
sudo sshd -T | grep -i maxauthtries
sudo sshd -T | grep -i x11forwarding
sudo sshd -T | grep -i allowusers
permitrootlogin no
passwordauthentication no
pubkeyauthentication yes
maxauthtries 3
x11forwarding yes
allowusers badis
```

sudo sshd -T | grep -i permitrootlogin
-i = ne tient pas en compte les majuscules et minuscules
-t = vérifie qu'il n'a pas d'erreur

1.2.6 : Test de survie

```
C:\Users\badis>ssh root@10.0.0.10
root@10.0.0.10: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

```
C:\Users\badis>ssh test@10.0.0.10
test@10.0.0.10: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

```
C:\Users\badis>ssh badis@10.0.0.10
Enter passphrase for key 'C:\Users\badis/.ssh/id_ed25519':
Register this system with Red Hat Insights: rhc connect
```

```
C:\Users\badis>ssh -o PubkeyAuthentication=no badis@10.0.0.10
badis@10.0.0.10: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
```

Obligé d'avoir la paraphrase

1.2.7 : Lire le journal

```
[badis@rhel9-lab ~]$ sudo journalctl -u sshd
```

```
[badis@rhel9-lab ~]$ sudo tail -n 30 /var/log/secure
Oct  8 15:58:12 rhel9-lab sshd-session[3057]: Received disconnect from 10.0.0.1 port 50509:11: disconnected by user
Oct  8 15:58:12 rhel9-lab sshd-session[3057]: Disconnected from user badis 10.0.0.1 port 50509
Oct  8 15:58:12 rhel9-lab sshd-session[3054]: pam_unix(sshd:session): session closed for user badis
```

le tail -n 30 = affiche les 30 dernières lignes du fichier.
Pour trouver le compte refusé (root)

```
[badis@rhel9-lab ~]$ sudo journalctl -u sshd | grep -i "not allowed"
oct. 08 15:58:25 rhel9-lab sshd-session[3091]: User root from 10.0.0.1 not allowed because not listed in AllowUsers
oct. 08 16:05:58 rhel9-lab sshd-session[3170]: User test from 10.0.0.1 not allowed because not listed in AllowUsers
```

-u : fonctionne pour n'importe quel service, c'est juste pour cibler.

1.2 : Point de contrôle

```
AllowUsers badis test
```

```
[badis@rhel9-lab ~]$ sudo sshd -T | grep -i allowusers
allowusers badis
allowusers test
```

```
[badis@rhel9-lab ~]$ ssh test@10.0.0.10
Enter passphrase for key '/home/badis/.ssh/id_ed25519':
Register this system with Red Hat Insights: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
[test@rhel9-lab ~]$
```

la paraphrase : [redacted]

Questions:

Q1.4 Pourquoi valider la syntaxe du fichier de configuration avant de recharger le service ?

Pour vérifier qu'il n'a pas d'erreur et éviter de perdre la main si on travaille à distance et qu'on a pas le poste physique.

Q1.5 Changer le port SSH de 22 à 2222 améliore-t-il la sécurité ? Justifiez.

Oui car par défaut le port SSH est 22, pour plus de sécurité contre les attaques par bots qui scannent automatiquement le port 22 (wireshark)

Q1.6 Quelle information apporte sshd -T que la lecture du fichier ne donne pas ?

Il affiche toutes les options, en gros ce que sshd applique réellement et évite les conflits.

TP 1.3 : Dix comptes, trois rôles, permissions et ACL

1.3.1 : Créer 3 groupes admin, dev, guest

```
admin:x:1002:
dev:x:1003:
guest:x:1004:
```

```
[badis@rhel9-lab ~]$ sudo groupadd admin
groupadd: group 'admin' already exists
[badis@rhel9-lab ~]$ sudo groupadd guest
groupadd: group 'guest' already exists
[badis@rhel9-lab ~]$ sudo groupadd dev
groupadd: group 'dev' already exists
```

1.3.2 : Créer 2 admin, 5 dev, 3 guest

```
[badis@rhel9-lab ~]$ getent group admin dev guest wheel
admin:x:1002:karim,david
dev:x:1003:lucas,ema,badise,gege,ahmed
guest:x:1004:yacine,luffy,ipman
wheel:x:10:badis,karim,david
```

wheel : groupe administrateurs sur Red Hat
Je

1.3.3 : Créer l'arborescence : mkdir

```
[badis@rhel9-lab ~]$ sudo mkdir -p /srv/{equipe,public,confidentiel}
[badis@rhel9-lab ~]$ sudo chown root:dev   /srv/equipe
sudo chown root:admin /srv/public
sudo chown root:admin /srv/confidentiel
```

Et maintenant faut définir les permissions.

Continuité sur Ubuntu :

TP 1.4 Un service systemd maison et ses journaux.

Ce programme est de demander à **systemd** de le gérer sous un compte dédié `monapp`, puis de planifier son exécution avec un timer.

```
#!/bin/bash

echo "=== MonApp ==="
echo "Date : $(date)"
echo "Espace disque :"
df -h /
```

J'ai séparé les éléments de l'application :

/srv/monapp/

└── monapp.sh ← programme de l'application

```
ubuntu@ubuntu-server:~$ sudo bash /srv/monapp.sh
=== MonApp ===
Date : Thu Oct  8 14:21:45 UTC 2026
Espace disque :
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   20G  1.9G   17G  10% /
```

verification du script monapp.sh

```
ubuntu@ubuntu-server:~$ sudo useradd --system --no-create-home --shell /usr/sbin/nologin monapp
ubuntu@ubuntu-server:~$ sudo mkdir -p /srv/monapp
ubuntu@ubuntu-server:~$ sudo mkdir -p /var/log/monapp
ubuntu@ubuntu-server:~$ sudo chown monapp:monapp /srv/monapp /var/log/monapp
ubuntu@ubuntu-server:~$ ls -ld /srv/monapp /var/log/monapp
drwxr-xr-x 2 monapp monapp 4096 Oct  8 14:26 /srv/monapp
drwxr-xr-x 2 monapp monapp 4096 Oct  8 14:26 /var/log/monapp
```

Un compte système `monapp` a été créé ainsi que les répertoires `/srv/monapp` et `/var/log/monapp`. Ces répertoires appartiennent au compte `monapp` afin que le service puisse fonctionner avec ce compte dédié.

```
ubuntu@ubuntu-server:~$ sudo mv /srv/monapp.sh /srv/monapp/monapp.sh
ubuntu@ubuntu-server:~$ ls -l /srv/monapp/
total 4
-rw-r--r-- 1 root root 88 Oct  8 14:19 monapp.sh
ubuntu@ubuntu-server:~$ sudo -u monapp bash /srv/monapp/monapp.sh
=== MonApp ===
Date : Thu Oct  8 14:34:50 UTC 2026
Espace disque :
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   20G  1.9G   17G  10% /
```

L'image me confirme que le compte monapp peut exécuter le script

```
  GNU nano 7.2
[Unit]
Description=Service MonApp
Wants=network-online.target
After=network-online.target

[Service]
Type=oneshot
User=monapp
Group=monapp
ExecStart=/bin/bash /srv/monapp/monapp.sh
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

L'unité `monapp.service` a été créée avec systemd. e service utilise le compte dédié `monapp`, dépend de la disponibilité du réseau et possède une politique de redémarrage automatique

```
ubuntu@ubuntu-server:~$ sudo systemctl daemon-reload
ubuntu@ubuntu-server:~$ sudo systemctl enable --now monapp.service
Created symlink /etc/systemd/system/multi-user.target.wants/monapp.service → /etc/systemd/system/monapp.service.
ubuntu@ubuntu-server:~$ systemctl status monapp.service
○ monapp.service - Service MonApp
     Loaded: loaded (/etc/systemd/system/monapp.service; enabled; preset: enabled)
     Active: inactive (dead) since Thu 2026-10-08 16:55:38 CEST; 23s ago
    Process: 1879 ExecStart=/bin/bash /srv/monapp/monapp.sh (code=exited, status=0/SUCCESS)
   Main PID: 1879 (code=exited, status=0/SUCCESS)
        CPU: 11ms

Oct 08 16:55:38 ubuntu-server systemd[1]: Starting monapp.service - Service MonApp...
Oct 08 16:55:38 ubuntu-server bash[1879]: === MonApp ===
Oct 08 16:55:38 ubuntu-server bash[1879]: Date : Thu Oct  8 16:55:38 CEST 2026
Oct 08 16:55:38 ubuntu-server bash[1879]: Espace disque :
Oct 08 16:55:38 ubuntu-server bash[1884]: Filesystem                         Size  Used Avail Use% Mounted on
Oct 08 16:55:38 ubuntu-server bash[1884]: /dev/mapper/ubuntu--vg-ubuntu--lv   20G  1.9G   17G  10% /
Oct 08 16:55:38 ubuntu-server systemd[1]: monapp.service: Deactivated successfully.
Oct 08 16:55:38 ubuntu-server systemd[1]: Finished monapp.service - Service MonApp.
```

Après avoir reload et démarré l'unité, le status me confirme le succès.
La commande `systemctl status monapp.service` confirme que le service a été correctement pris en charge par systemd. Le script s'est exécuté, a affiché la date et l'espace disque disponible, puis s'est terminé avec le code `0`, indiquant une exécution réussie.

```
GNU nano 7.2
[Unit]
Description=Timer MonApp

[Timer]
OnCalendar=*-*-* *:*:00
Persistent=true

[Install]
WantedBy=timers.target
```

Avec `systemctl list-timers --all`, on peut vérifier que le timer `monapp.timer` est bien enregistré par systemd

```
ubuntu@ubuntu-server: $ systemctl status monapp.timer
● monapp.timer - Timer MonApp
     Loaded: loaded (/etc/systemd/system/monapp.timer; enabled; preset: enabled)
     Active: active (waiting) since Thu 2026-10-08 20:07:26 CEST; 10min ago
    Trigger: Thu 2026-10-08 20:19:00 CEST; 34s left
   Triggers: ● monapp.service

Oct 08 20:07:26 ubuntu-server systemd[1]: Started monapp.timer - Timer MonApp.
ubuntu@ubuntu-server: $ systemctl list-timers --all
NEXT                         LEFT LAST                         PASSED UNIT                         ACTIVATES
Thu 2026-10-08 20:19:00 CEST      30s Thu 2026-10-08 20:18:20 CEST 8s ago monapp.timer              monapp.service
Thu 2026-10-08 20:22:20 CEST 3min 51s -                               - systemd-tmpfiles-clean.timer systemd-tmpfiles-clean.service
Thu 2026-10-08 22:20:02 CEST  2h 1min Thu 2026-10-08 16:39:15 CEST      - apt-daily.timer              apt-daily.service
Fri 2026-10-09 00:00:00 CEST 3h 41min Thu 2026-10-08 09:16:07 CEST      - dpkg-db-backup.timer         dpkg-db-backup.service
Fri 2026-10-09 01:41:55 CEST 5h 23min Wed 2026-10-07 09:38:06 CEST      - man-db.timer                 man-db.service
Fri 2026-10-09 06:20:10 CEST      10h Thu 2026-10-08 16:55:38 CEST      - motd-news.timer              motd-news.service
Fri 2026-10-09 06:48:02 CEST      10h Thu 2026-10-08 09:57:11 CEST      - apt-daily-upgrade.timer      apt-daily-upgrade.service
Sun 2026-10-11 03:10:30 CEST   2 days Wed 2026-10-07 09:36:02 CEST      - e2scrub_all.timer            e2scrub_all.service
Mon 2026-10-12 00:00:41 CEST   3 days Wed 2026-10-07 09:36:02 CEST      - fstrim.timer                 fstrim.service
-                                    - -                              - apport-autoreport.timer      apport-autoreport.service
-                                    - -                              - snapd.snap-repair.timer      snapd.snap-repair.service

11 timers listed.
```

la premiere commande assure que l'unité a bien été activée.
La deuxième est bien enregistrée par systemd et affiche sa prochaine exécution.

```
Date : Thu Oct  8 20:19:28 CEST 2026
Espace disque :
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   20G  1.9G   17G  10% /
monapp.service: Deactivated successfully.
Finished monapp.service - Service MonApp.
Starting monapp.service - Service MonApp...
=== MonApp ===
Date : Thu Oct  8 20:20:28 CEST 2026
Espace disque :
Filesystem                         Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv   20G  1.9G   17G  10% /
monapp.service: Deactivated successfully.
Finished monapp.service - Service MonApp.
Starting monapp.service - Service MonApp...
```

Puis la vérification du timer sur le monapp.service toutes les minutes à peu près, comme indiqué dans le précédent screen, on le voit grâce à cette commande :
$journalctl -u monapp.service --since today

1.5 Synthèse et vérifications

```
ubuntu@ubuntu-server:~$ sudo vgs
  VG        #PV #LV #SN Attr   VSize   VFree
  ubuntu-vg   1   5   0 wz--n- <48.00g <6.00g
```

Le volume groupe a un espace libre de 6 Giga.

```
ubuntu@ubuntu-server:~$ sudo lvextend -l +100%FREE /dev/ubuntu-vg/log
  Size of logical volume ubuntu-vg/log changed from 5.00 GiB (1280 extents) to <11.00 GiB (2815 extents).
  Logical volume ubuntu-vg/log successfully resized.
```

Cela agrandit seulement LV (logic volume) /log, qui fait maintenant 11 Giga.

```
ubuntu@ubuntu-server:~$ sudo resize2fs /dev/ubuntu-vg/log
resize2fs 1.47.0 (5-Feb-2023)
Filesystem at /dev/ubuntu-vg/log is mounted on /var/log; on-line resizing required
old_desc_blocks = 1, new_desc_blocks = 2
The filesystem on /dev/ubuntu-vg/log is now 2882560 (4k) blocks long.
```

Il faut ensuite agrandir le système de fichiers.

```
ubuntu@ubuntu-server:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0   50G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0    1G  0 part /boot
└─sda3                      8:3    0   48G  0 part
  ├─ubuntu--vg-ubuntu--lv 252:0    0   20G  0 lvm  /
  ├─ubuntu--vg-var        252:1    0   10G  0 lvm  /var
  ├─ubuntu--vg-log        252:2    0   11G  0 lvm  /var/log
  ├─ubuntu--vg-home       252:3    0    5G  0 lvm  /home
  └─ubuntu--vg-swap       252:4    0    2G  0 lvm  [SWAP]
sr0                        11:0    1  3.8G  0 rom
```

Pour vérifier le tout, le système de fichiers ext4 a bien été agrandi.

```
#!/bin/bash

echo "========================================="
echo "     AUDIT DE DURCISSEMENT - TP 1.5"

echo
echo "1. PARTITIONNEMENT / VAR ET /VAR/LOG"
echo "-----------------------------------------"
lsblk -f
echo
findmnt /var
findmnt /var/log

echo
echo "2. ESPACE LIBRE DANS LE GROUPE LVM"
echo "-----------------------------------------"
sudo vgs

echo
echo "3. MISES A JOUR"
echo "-----------------------------------------"
apt list --upgradable 2>/dev/null

echo
echo "4. SERVICES EN ECHEC"
echo "-----------------------------------------"
systemctl --failed

echo
echo "5. COMPTE ROOT"
echo "-----------------------------------------"
sudo passwd -S root

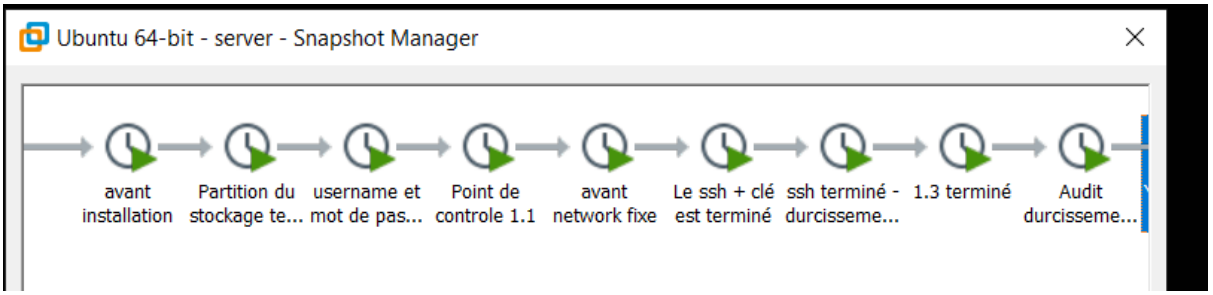
echo
echo "6. CONFIGURATION SSH"
echo "-----------------------------------------"
sudo sshd -T | grep -Ei 'permitrootlogin|passwordauthentication|pubkeyauthentication|allowusers|maxauthtries'

echo
echo "7. COMPTES SANS MOT DE PASSE"
echo "-----------------------------------------"
sudo awk -F: '$2==""*" {print $1}' /etc/shadow


echo
echo "8. CONFIGURATION RESEAU"
echo "-----------------------------------------"
ip -br a
ip route

echo
```

Afin de faire la check-list de sécurité du serveur, j'ai fait un script qui regroupe directement toutes les commandes nécessaires.
Avec sudo bash ~/audit_durcissement.sh



Voici les différents snapshots.

TP 2.1 : Script 1 : Création d’utilisateurs en lot depuis un fichier CSV

Pour ce TP 2.1, on va le faire d’une autre manière. On va faire un menu interactif où on va décider de sélectionner quel script on veut exécuter :

déjà définir les étapes pour le menu :

1 : Créer le fichier nouveaux.csv : 1. Fichier d'entrée avec ligne de commentaire en tête
2 : Afficher le fichier CSV : Contrôle visuel avant traitement
3 : Vérifier le fichier (sans créer) : 2 et 3. Lecture et validation sans toucher au système
4 : Créer les comptes : 2,3 et 4. Lecture validation et création idempotente
5 : Afficher le récapitulatif : 5 et 6. Compteurs et code de sortie
6 Exit : Quitter

On va créer le script.sh en bash.

```
[badis@rhel9-lab ~]$ touch mon_script.sh
```

J’ai défini que on puisse l'exécuter puis après je vais définir les permissions pour que user badis puisse l'exécuter.

```
[badis@rhel9-lab ~]$ chmod 700 mon_script.sh
```

Seul badis a accès pour modifier exécuter et lire.

Informations important à connaître pour pouvoir faire le menu :

break : sort de la boucle select
; ; termine chaque case
*) attrape tout numéro qui n’est pas dans la liste
$REPLY : ce que l'utilisateur a tapé (chiffre) et case $valeur in : regarde ce que contient $valeur

On a défini les bases du script:

```bash
#!/bin/bash
# Menu interactif - Création de comptes en masse

PS3="Quelle tâche souhaitez-vous exécuter ? "

options=(
  "Créer le fichier nouveaux.csv"
  "Afficher le fichier CSV"
  "Vérifier le fichier (sans créer)"
  "Créer les comptes"
  "Afficher le récapitulatif"
  "Exit"
)

select choix in "${options[@]}"; do
  case $REPLY in
    1) # créer le fichier CSV
       echo "À faire : créer le fichier"
       ;;
    2) # afficher le fichier
       echo "À faire : afficher le fichier"
       ;;
    3) # vérifier sans créer
       echo "À faire : vérifier le fichier"
       ;;
    4) # créer les comptes
       echo "À faire : créer les comptes"
       ;;
    5) # afficher le récapitulatif
       echo "À faire : afficher le récapitulatif"
       ;;
    6) echo "Au revoir"
       break
       ;;
    *) echo "Choix invalide"
       ;;
  esac
```

On a exécuté le menu pour voir si le menu fonctionne bien avant de commencer sur les tâches à faire.

```
[badis@rhel9-lab ~]$ ./mon_script.sh
1) Créer le fichier nouveaux.csv     3) Vérifier le fichier (sans créer)  5) Afficher le récapitulatif
2) Afficher le fichier CSV           4) Créer les comptes                 6) Exit
Quelle tâche souhaitez-vous exécuter ? 1
À faire : créer le fichier
Quelle tâche souhaitez-vous exécuter ? 2
À faire : afficher le fichier
Quelle tâche souhaitez-vous exécuter ? 3
À faire : vérifier le fichier
Quelle tâche souhaitez-vous exécuter ? 4
À faire : créer les comptes
Quelle tâche souhaitez-vous exécuter ? 5
À faire : afficher le récapitulatif
Quelle tâche souhaitez-vous exécuter ? 9
Choix invalide
Quelle tâche souhaitez-vous exécuter ? 6
Au revoir
[badis@rhel9-lab ~]$
```

```bash
    1) # créer le fichier CSV
       mkdir -p /srv/scripts
       cat > "$FICHIER" << 'EOF'
# nom,groupe,role
dev6,dev,developpeur
guest4,guest,invite
ops3,admin,administrateur
EOF
       echo "Fichier $FICHIER créé."
       ;;*
```

cat copie les valeurs pour $FICHIER (variable) et il s'arrête

dès qu'il voit EOF comme valeur.

```
[badis@rhel9-lab ~]$ sudo ./mon_script.sh
1) Créer le fichier nouveaux.csv     3) Vérifier le fichier (sans créer)  5) Afficher le récapitulatif
2) Afficher le fichier CSV           4) Créer les comptes                 6) Exit
Quelle tâche souhaitez-vous exécuter ? 1
Fichier /srv/scripts/nouveaux.csv créé.
Quelle tâche souhaitez-vous exécuter ? 6
Au revoir
```

Le fichier /srv/scripts/nouveaux.csv a était crée :

```
[badis@rhel9-lab ~]$ sudo cat /srv/scripts/nouveaux.csv
# nom,groupe,role
dev6,dev,developpeur
guest4,guest,invite
ops3,admin,administrateur
```

Option 2 : Afficher le fichier csv

```
Quelle tâche souhaitez-vous exécuter ? 2
     1  # nom,groupe,role
     2  dev6,dev,developpeur
     3  guest4,guest,invite
     4  ops3,admin,administrateur
```

```
2) # afficher le fichier
   if [[ -f "$FICHIER" ]]; then
      cat -n "$FICHIER"
   else
    echo "Le fichier $FICHIER n'existe pas, utilisée d'abord l'option 1."
   fi
   ;;
```

Option 3 :

```
3) # vérifier sans créer
   while IFS=, read -r nom groupe role; do
     [[ -z $nom || $nom == \#* ]] && continue
     if   [[ -z $role ]];                    then echo "REJET  $nom : il manque un champ"
     elif [[ $nom == *" "* ]];               then echo "REJET  $nom : nom invalide"
     elif ! getent group $groupe > /dev/null; then echo "REJET  $nom : groupe inexistant"
     elif id $nom &> /dev/null;              then echo "IGNORE $nom : existe déjà"
     else echo "OK     $nom : sera créé"
     fi
   done < "$FICHIER"
   ;;
```

```
[badis@rhel9-lab ~]$ ./mon_script.sh
1) Créer le fichier nouveaux.csv     3) Vérifier le fichier (sans créer)  5) Afficher le récapitulatif
2) Afficher le fichier CSV           4) Créer les comptes                 6) Exit
Quelle tâche souhaitez-vous exécuter ? 3
OK     dev6 : sera créé
OK     guest4 : sera créé
OK     ops3 : sera créé
```

Option 4 5 6:

```
4) # créer les comptes
   crees=0; ignores=0; rejets=0
   while IFS=, read -r nom groupe role; do
     [[ -z $nom || $nom == \#* ]] && continue
     if   [[ -z $role ]];                  then echo "REJET  $nom : il manque un champ"; rejets=$((rejets+1))
     elif [[ $nom == *" "* ]];             then echo "REJET  $nom : nom invalide";       rejets=$((rejets+1))
     elif ! getent group $groupe > /dev/null; then echo "REJET  $nom : groupe inexistant"; rejets=$((rejets+1))
     elif id $nom &> /dev/null;            then echo "IGNORE $nom : existe déjà";        ignores=$((ignores+1))
     else
       useradd -m -s /bin/bash -G $groupe -c "$role" $nom
       echo "CREE   $nom ($groupe)"; crees=$((crees+1))
     fi
   done < "$FICHIER"
   ;;
5) # afficher le récapitulatif
   echo "Créés   : $crees"
   echo "Ignorés : $ignores"
   echo "Rejetés : $rejets"
   ;;
6) echo "Au revoir"
   break
   ;;
*) echo "Choix invalide"
   ;;
```

Créer les comptes avec un compteur pour voir ce qui a était créer, ignorer, et les rejets sa te liste aussi les group

REJET = une option que tu définis pour voir s'il manque un champ, nom invalide, groupe inexistant, existe déjà.

et sa définis useradd dans son group et le rôle.
et break = "sortir" du script

voilà le résultat :

```
[badis@rhel9-lab ~]$ sudo ./mon_script.sh
[sudo] Mot de passe de badis :
1) Créer le fichier nouveaux.csv     3) Vérifier le fichier (sans créer)  5) Afficher le récapitulatif
2) Afficher le fichier CSV           4) Créer les comptes                 6) Exit
Quelle tâche souhaitez-vous exécuter ? 1
Fichier /srv/scripts/nouveaux.csv créé.
Quelle tâche souhaitez-vous exécuter ? 2
     1  # nom,groupe,role
     2  dev6,dev,developpeur
     3  guest4,guest,invite
     4  ops3,admin,administrateur
Quelle tâche souhaitez-vous exécuter ? 3
IGNORE dev6 : existe déjà
IGNORE guest4 : existe déjà
IGNORE ops3 : existe déjà
Quelle tâche souhaitez-vous exécuter ? 4
IGNORE dev6 : existe déjà
IGNORE guest4 : existe déjà
IGNORE ops3 : existe déjà
Quelle tâche souhaitez-vous exécuter ? 5
Créés   : 0
Ignorés : 3
Rejetés : 0
Quelle tâche souhaitez-vous exécuter ? 6
Au revoir
```

Q2.1 Pourquoi la redirection du fichier se place-t-elle après done et non après read ?

Done signifie la fin de la boucle, voilà la raison pour laquelle.
read : lit la ligne suivante et si on le mets à côté d'un while = boucle infini

Q2.2 Qu'est-ce qu'un script idempotent, et pourquoi est-ce indispensable sur un parc de cent serveurs ?

Un script est idempotent quand on peut l'exécuter une fois ou plusieurs fois avec le même résultat final et aussi car le déploiement n'est jamais parfait avec beaucoup de variable dans un parc de cent serveurs.

Q2.3 Pourquoi une ligne invalide ne doit-elle pas interrompre le traitement des suivantes ?

Elle ne doit pas interrompre le traitement des suivantes car sinon le script ne sera pas utilisable et il s'arrête à l'étape invalide

TP 2.2 : Script 2 : Création d'utilisateurs en lot depuis un fichier CSV

Le plan est toujours le même :

```bash
#!/bin/bash
#Script Santé : Menu interactif + compteur d'alertes.
SEUIL_CHARGE=1
SEUIL_DISQUE=80
SEUIL_INODES=80
SEUIL_MEMOIRE=15
ALERTES=0

menu() {
  PS3="Quelle vérification souhaitez-vous exécuter ? "

  options=(
    "Contrôler la charge système"
    "Contrôler l'espace disque"
    "Contrôler les inodes"
    "Contrôler la mémoire"
    "Lister les unités systemd en échec"
    "Rapport complet"
    "Afficher les seuils"
    "Exit"
  )

  select choix in "${options[@]}"; do
    case $REPLY in
      1) verifier_charge ;;
      2) verifier_disque ;;
      3) verifier_inodes ;;
      4) verifier_memoire ;;
      5) verifier_systemd ;;
      6) rapport_complet ;;
      7) echo "Charge  : $SEUIL_CHARGE x nombre de CPU"
         echo "Disque  : $SEUIL_DISQUE %"
         echo "Inodes  : $SEUIL_INODES %"
         echo "Mémoire : $SEUIL_MEMOIRE % disponible minimum"
         ;;
      8) echo "Au revoir ($ALERTES alerte(s))"
         if [[ $ALERTES -gt 0 ]]; then exit 1; else exit 0; fi
         ;;
      *) echo "Choix invalide" ;;
    esac
  done
}

menu   #lance le menu
```

On a ajouté un compteur d'alertes

1 à 6 on a mis des fonctions pour que le code soit pas horrible à lire.

Option 1 :

```
[sudo] Mot de passe de badis :
1) Contrôler la charge système        3) Contrôler les inodes              5) Lister les unités systemd en échec  7) Afficher les seuils
2) Contrôler l'espace disque          4) Contrôler la mémoire              6) Rapport complet                     8) Exit
Quelle vérification souhaitez-vous exécuter ? 1
OK     Charge : 0.00 pour 3 CPU (limite 3)
```

```bash
verifier_charge(){
    charge=$(cut -d' ' -f1 /proc/loadavg)
    entier=${charge%.*}
    cpu=$(nproc)
    limite=$((cpu * SEUIL_CHARGE)) #normalement 1

    if [[ $entier -ge $limite ]]; then
      echo "ALERTE Charge : $charge pour $cpu CPU (limite $limite)"
      ALERTES=$((ALERTES+1))
    else
      echo "OK     Charge : $charge pour $cpu CPU (limite $limite)"
    fi
}
```

1. on a défini le seuil de charge et ALERTES=0

on a défini les variables charge entier cpu limite

$: on lit ce qu'il y'a dedans

sans $ = on écrit une variable

Option 2 :

```
1) Contrôler la charge système        3) Contrôler les inodes              5) Lister les unités systemd en échec  7) Afficher les seuils
2) Contrôler l'espace disque          4) Contrôler la mémoire              6) Rapport complet                     8) Exit
Quelle vérification souhaitez-vous exécuter ? 2
OK     Disque : tous les volumes sous 80 %
Quelle vérification souhaitez-vous exécuter ? 8
Au revoir (0 alerte(s))
[badis@rhel9-lab ~]$ df -h
Sys. de fichiers      Taille Utilisé Dispo Uti% Monté sur
devtmpfs                1,8G       0  1,8G   0% /dev
tmpfs                   1,8G       0  1,8G   0% /dev/shm
tmpfs                   725M     11M  715M   2% /run
efivarfs                256K     57K  195K  23% /sys/firmware/efi/efivars
/dev/mapper/rhel-root    20G    1,9G   19G  10% /
/dev/nvme0n1p2          960M    488M  473M  51% /boot
/dev/nvme0n1p1          599M    7,7M  592M   2% /boot/efi
/dev/mapper/rhel-home    10G    104M  9,9G   2% /home
/dev/mapper/rhel-var     10G    640M  9,4G   7% /var
/dev/nvme0n1p4          4,0G     74M  3,9G   2% /var/log
tmpfs                   363M       0  363M   0% /run/user/1000
```

df -h pour vérifier si le disque est plein pour nous (humain)

```bash
verifier_disque() {
  probleme=0
  while read -r pourcent point; do
    pourcent=${pourcent%\%}
    if [[ $pourcent -gt $SEUIL_DISQUE ]]; then
      echo "ALERTE Disque : $point est rempli à $pourcent % (seuil $SEUIL_DISQUE %)"
      ALERTES=$((ALERTES+1))
      probleme=1
    fi
  done < <(df --output=pcent,target -x tmpfs -x devtmpfs | tail -n +2)

  if [[ $probleme -eq 0 ]]; then
    echo "OK     Disque : tous les volumes sous $SEUIL_DISQUE %"
  fi
}
```

-x c’est pour exclure les systèmes de fichiers type virtuels (tmpfs et devtmpfs)
et on a trié pour garder que les 2 colonnes les plus importants : le pourcentage utilisé et le point de montage (**pcent et target**)

Option 3 : Inode

inode : une fiche d’identité, qui contient son propriétaire, ses droits, sa taille et l’endroit où sont rangées ses données. le nombre d’inodes est limité durant la création de fichier.

```
[badis@rhel9-lab ~]$ sudo ./sante.sh
[sudo] Mot de passe de badis :
1) Contrôler la charge système        3) Contrôler les inodes             5) Lister les unités systemd en échec  7) Afficher les seuils
2) Contrôler l'espace disque          4) Contrôler la mémoire             6) Rapport complet                     8) Exit
Quelle vérification souhaitez-vous exécuter ? 3
OK     Inodes : tous les volumes sous 80 %
```

```bash
verifier_inodes() {
  probleme=0
  while read -r pourcent point; do
    pourcent=${pourcent%\%}
    if [[ $pourcent == "-" ]]; then
      continue
    fi
    if [[ $pourcent -gt $SEUIL_INODES ]]; then
      echo "ALERTE Inodes : $point utilise $pourcent % de ses inodes (seuil $SEUIL_INODES %)"
      ALERTES=$((ALERTES+1))
      probleme=1
    fi
  done < <(df --output=ipcent,target -x tmpfs -x devtmpfs | tail -n +2)

  if [[ $probleme -eq 0 ]]; then
    echo "OK     Inodes : tous les volumes sous $SEUIL_INODES %"
  fi
}
```

Option 4 :

```bash
verifier_memoire() {
  total=$(grep MemTotal /proc/meminfo | awk '{print $2}')
  dispo=$(grep MemAvailable /proc/meminfo | awk '{print $2}')
  pourcent=$((dispo * 100 / total))

  if [[ $pourcent -lt $SEUIL_MEMOIRE ]]; then
    echo "ALERTE Mémoire : $pourcent % disponible (seuil $SEUIL_MEMOIRE %)"
    ALERTES=$((ALERTES+1))
  else
    echo "OK     Mémoire : $pourcent % disponible (seuil $SEUIL_MEMOIRE %)"
  fi
}
```

```
[sudo] Mot de passe de badis :
1) Contrôler la charge système        3) Contrôler les inodes             5) Lister les unités systemd en échec  7) Afficher les seuils
2) Contrôler l'espace disque          4) Contrôler la mémoire             6) Rapport complet                     8) Exit
Quelle vérification souhaitez-vous exécuter ? 4
OK     Mémoire : 84 % disponible (seuil 15 %)
Quelle vérification souhaitez-vous exécuter ?
```

grep -E "MemTotal|MemAvailable" /proc/meminfo

```
[badis@rhel9-lab ~]$ grep -E "MemTotal|MemAvailable" /proc/meminfo
MemTotal:        3709404 kB
MemAvailable:    3141264 kB
```

Notre script le traduit en pourcentage

awk : lire ou choisit des colonnes et produire un résultat (filtre et calcule)

Option 5 :

```
[badis@rhel9-lab ~]$ sudo ./sante.sh
1) Contrôler la charge système        3) Contrôler les inodes              5) Lister les unités systemd en échec  7) Afficher les seuils
2) Contrôler l'espace disque          4) Contrôler la mémoire              6) Rapport complet                     8) Exit
Quelle vérification souhaitez-vous exécuter ? 5
OK     Systemd : aucune unité en échec
Quelle vérification souhaitez-vous exécuter ?
```

```
verifier_systemd() {
  nombre=$(systemctl --failed --no-legend | wc -l)

  if [[ $nombre -gt 0 ]]; then
    echo "ALERTE Systemd : $nombre unité(s) en échec :"
    systemctl --failed --no-legend
    ALERTES=$((ALERTES+1))
  else
    echo "OK     Systemd : aucune unité en échec"
  fi
}
```

systemctl –failed = vérifier si y’a un problème dans mes services

```
[badis@rhel9-lab ~]$ sudo ./sante.sh
1) Contrôler la charge système        3) Contrôler les inodes              5) Lister les unités systemd en échec  7) Afficher les seuils
2) Contrôler l'espace disque          4) Contrôler la mémoire              6) Rapport complet                     8) Exit
Quelle vérification souhaitez-vous exécuter ? 5
ALERTE Systemd : 1 unité(s) en échec :
● test-echec.service loaded failed failed /bin/false
```

On a créer un service qui échoue pour tester et ça fonctionne bien

Option 6 (final) :

```
Quelle vérification souhaitez-vous exécuter ? 6
===== Santé de rhel9-lab - 09/10/2026 13:54 ===
OK     Charge : 0.00 pour 3 CPU (limite 3)
OK     Disque : tous les volumes sous 80 %
OK     Inodes : tous les volumes sous 80 %
OK     Mémoire : 84 % disponible (seuil 15 %)
OK     Systemd : aucune unité en échec
=== Synthèse : 0 alerte(s) ===
```

```
verifier_systemd() {
  nombre=$(systemctl --failed --no-legend | wc -l)

  if [[ $nombre -gt 0 ]]; then
    echo "ALERTE Systemd : $nombre unité(s) en échec :"
    systemctl --failed --no-legend
    ALERTES=$((ALERTES+1))
  else
    echo "OK     Systemd : aucune unité en échec"
  fi
}
```

ALERTES=0 c'est juste pour remettre le compteur à 0 avant le rapport.
awk :

```
Quelle vérification souhaitez-vous exécuter ? 7
Charge  : 1 x nombre de CPU
Disque  : 80 %
Inodes  : 80 %
Mémoire : 15 % disponible minimum
```

```
 7) echo "Charge  : $SEUIL_CHARGE x nombre de CPU"
    echo "Disque  : $SEUIL_DISQUE %"
    echo "Inodes  : $SEUIL_INODES %"
    echo "Mémoire : $SEUIL_MEMOIRE % disponible minimum"
    ;;
```

Q2.4 — Pourquoi comparer la charge au nombre de processeurs plutôt qu'à une valeur fixe ?
Pour nous indiquer combien de processus veulent utiliser le processeur. c une alterne pour toutes les machines.

Q2.5 — Que signifie une alerte sur les inodes alors que l'espace disque est disponible ?

La partition est mal faite malgré le fait d'avoir de l'espace de disque disponible.
chaque fichier utilise des inode et le nombre est limité et on pourras plus en créer de nouveaux fichiers si y'a une alerte sur les inodes voir avec df -i

Q2.6 — Pourquoi un script de vérification doit-il rendre un code de sortie non nul ? À qui sert cette information
Un programme renvoie 0 quand il va bien et une valeur non nul en cas de problème.
L'information sert à la machine ou des outils comme cron

TP 2.3 : Script 3 : Sauvegarde avec compression et rotation

```bash
#!/bin/bash

#valeur
SOURCE=""
DEST=""
COMPRESSION=0     # 0 = non, 1 = oui
RETENTION=7       # nombre d'archives à garder
VERBEUX=0         # 0 = non, 1 = oui

PS3="Votre choix : "
select option in "-s source" "-d destination" "-c compression" "-r retention" "-v verbeux" "-h aide" "Quitter"; do
    case "$option" in
        "-s source")      read -rp "Dossier source : " SOURCE ;;
        "-d destination") read -rp "Dossier destination : " DEST ;;
        "-c compression") COMPRESSION=1; echo "Compression activée" ;;
        "-r retention")   read -rp "Nombre d'archives à garder : " RETENTION ;;
        "-v verbeux")     VERBEUX=1; echo "Mode verbeux activé" ;;
        "-h aide")        echo "Choisissez une option avec son numéro" ;;
        "Quitter")        break ;;
        *)                echo "Choix invalide" ;;
    esac
done
```

```
1) -s source
2) -d destination
3) -c compression
4) -r retention
5) -v verbeux
6) -h aide
7) Quitter
Votre choix :
```

Q2.7 — Pourquoi vérifier l'espace disponible avant de copier plutôt que d'échouer en cours de route ?
Si le disque se remplit pendant la copie, on a une archive incomplète et inutilisable.

Q2.8 — Pourquoi le nettoyage doit-il être dans un piège plutôt qu'à la fin du script ?
Si le script s’arrête avant la fin en gros (erreur contrôle C), le nettoyage à la fin n’est jamais exécuté

Q2.9 — Quelle différence entre une sauvegarde et un instantané de machine virtuelle ?
Un instantané permet de revenir rapidement à un état précédent, mais il est stocké avec la VM, si la vm perdu alors on perds tout, alors que une sauvegarde on récupère l’intégralité.

TP 3 :

3.1 awk : analyser les comptes du système

```
ubuntu@ubuntu-server:~$ awk -F: '$3 >= 1000 {printf "%-14s uid=%-6d gid=%-6d %s\n", $1, $3, $4, $6}' /etc/passwd
nobody         uid=65534  gid=65534  /nonexistent
ubuntu         uid=1000   gid=1000   /home/ubuntu
admin          uid=1001   gid=1001   /home/admin
administrateur1 uid=1002   gid=1001   /home/administrateur1
administrateur2 uid=1003   gid=1001   /home/administrateur2
dev1           uid=1004   gid=1002   /home/dev1
dev2           uid=1005   gid=1002   /home/dev2
dev3           uid=1006   gid=1002   /home/dev3
dev4           uid=1007   gid=1002   /home/dev4
dev5           uid=1008   gid=1002   /home/dev5
guest1         uid=1009   gid=1003   /home/guest1
guest2         uid=1010   gid=1003   /home/guest2
guest3         uid=1011   gid=1003   /home/guest3
```

Pour extraire, avec awk, les infos de la liste des comptes pour identifiant supérieur ou égal à 1000.

pour comprendre la syntaxe %d = afficher un nombre entier, 6 = réserver 6 caractères et - = aligner à gauche. puis va chercher les informations dans le chemin etc/passwd

```
ubuntu@ubuntu-server:~$ sudo cat /etc/shadow
root:*:20705:0:99999:7:::
daemon:*:20705:0:99999:7:::
bin:*:20705:0:99999:7:::
sys:*:20705:0:99999:7:::
sync:*:20705:0:99999:7:::
games:*:20705:0:99999:7:::
man:*:20705:0:99999:7:::
lp:*:20705:0:99999:7:::
mail:*:20705:0:99999:7:::
news:*:20705:0:99999:7:::
```

Le fichier /etc/shadow contient notamment les empreintes/mots de passe des comptes.
Afin de savoir comment est continué le fichier, je modifie ma ligne de commande en ajoutant “*” au lieu de “ ”

```
ubuntu@ubuntu-server:~$ sudo awk -F: '$2 == "*" {print "sans mot de passe :", $1}' /etc/shadow
sans mot de passe : root
sans mot de passe : daemon
sans mot de passe : bin
sans mot de passe : sys
sans mot de passe : sync
sans mot de passe : games
sans mot de passe : man
sans mot de passe : lp
sans mot de passe : mail
sans mot de passe : news
sans mot de passe : uucp
sans mot de passe : proxy
sans mot de passe : www-data
sans mot de passe : backup
sans mot de passe : list
sans mot de passe : irc
sans mot de passe : _apt
sans mot de passe : nobody
```

Ceci m'affiche les comptes sans mdp : **awk -F: '$2 == "" {print "sans mot de passe :", $1}' /etc/shadow**

```
ubuntu@ubuntu-server:~$ awk -F: '{compte[$7]++} END {for (s in compte) print compte[s], s}' /etc/passwd | sort -rn
24 /usr/sbin/nologin
13 /bin/bash
2 /bin/false
1 /bin/sync
```

La commande va regarder **le 7e champ de `/etc/passwd`**, qui correspond au **shell de connexion**. Cette analyse permet de distinguer les comptes pouvant ouvrir une session.

```
ubuntu@ubuntu-server:~$ awk -F: '$3 >= 1000 {print $1, $6}' /etc/passwd | while read -r u h; do
>     [[ -d $h ]] || echo "home absent : $u ($h)"
> done
home absent : nobody (/nonexistent)
```

La commande awk sur ce screenshot repère les comptes dont le répertoire personnel n'existe pas. Le $3 et le 1000 viennent d'une convention des UID sur Linux qui démarre à 1000 pour les users.

Le compte `nobody` avec le répertoire `/nonexistent`, et que ce répertoire **n'existe pas**. C'est un **compte système**, utilisé par certains services pour fonctionner avec très peu de privilèges.

```
ubuntu@ubuntu-server:~$ awk -F: '
> BEGIN {
>     printf "%-14s %-8s %-8s %s\n", "COMPTE", "UID", "GID", "HOME"
> }
> $3 >= 1000 {
>     printf "%-14s %-8d %-8d %s\n", $1, $3, $4, $6
> }
> ' /etc/passwd
COMPTE         UID      GID      HOME
nobody         65534    65534    /nonexistent
ubuntu         1000     1000     /home/ubuntu
admin          1001     1001     /home/admin
administrateur1 1002     1001     /home/administrateur1
administrateur2 1003     1001     /home/administrateur2
dev1           1004     1002     /home/dev1
dev2           1005     1002     /home/dev2
dev3           1006     1002     /home/dev3
dev4           1007     1002     /home/dev4
dev5           1008     1002     /home/dev5
guest1         1009     1003     /home/guest1
guest2         1010     1003     /home/guest2
guest3         1011     1003     /home/guest3
```

Une sortie propre des valeurs. BEGIN va exécuter une seule fois avant de lire /etc/passwd.

Question 3.1 : NF = nombre de champs et NR = numéro de ligne. Le dernier champ s'obtient avec NF

Question 3.2 : awk a une syntaxe, enlever les guillemets empêcherait $1, $3, et ce qui est en dehors des crochets.

Question 3.3 : Un compte peut exister sans répertoire personnel.

3.2 sed : modifier des configurations en série

```
ubuntu@ubuntu-server:~$ ubuntu@ubuntu-server:~$ cat config.yml
port: 8080
# port: 8080
transport: tcp
```

Le fichier `config.yml` a été créé, ces trois éléments vont permettre de vérifier la substitution avec `sed`.

```
ubuntu@ubuntu-server:~$ sed 's/^\(\s*port:\s*\)[0-9]\+/\18443/' config.yml | diff - config.yml
1c1
< port: 8443
---
> port: 8080
```

`Il compare le résultat prévu (8080) avec le fichier original (8443) sur le champ port, sans le modifier.`

```
ubuntu@ubuntu-server:~$ sed -i.bak -E 's/^(\s*port:\s*)8080/\18443/' config.yml
ubuntu@ubuntu-server:~$ cat config.yml.bak
port: 8443
# port: 8080
transport: tcp
ubuntu@ubuntu-server:~$
```

On voit un fichier original, on l'a copié dans un backup et on a mis les nouvelles modifs dans le fichier principal.

```
ubuntu@ubuntu-server:~/config-test$ ls
config1.yml  config2.yml
```

Je crée les fichiers de configuration, afin de faire les tests.

```
ubuntu@ubuntu-server:~/config-test$ grep -rl 'port: 8080' ~/config-test/ | tee /tmp/liste.txt
/home/ubuntu/config-test/config2.yml
/home/ubuntu/config-test/config1.yml
```

Il faut lister les fichiers ayant 8080. **`tee`** sert à la fois à afficher le résultat dans le terminal ET à l'enregistrer dans un fichier

```
ubuntu@ubuntu-server:~/config-test$ sudo cat /tmp/liste.txt
/home/ubuntu/config-test/config2.yml
/home/ubuntu/config-test/config1.yml
ubuntu@ubuntu-server:~/config-test$ cat /home/ubuntu/config-test/config2.yml
port: 8443
# port: 8080
transport: tcp
```

La liste a bien été appliquée dans le fichier .txt.

```
ubuntu@ubuntu-server:~/config-test$ xargs -a /tmp/liste.txt -r sed -i.bak -E 's/^(\s*port:\s*)8080/\18443/'
ubuntu@ubuntu-server:~/config-test$ cat config1.yml
port: 8443
# port: 8080
transport: tcp
```

J'applique la même opération sur tout un répertoire.
Le cat affiche bien que 8080 a été mis en commentaire et a été remplacé par 8443.

```
ubuntu@ubuntu-server:~/config-test$ for f in $(cat /tmp/liste.txt); do mv -f "$f.bak" "$f"; done
ubuntu@ubuntu-server:~/config-test$ ls
config1.yml  config2.yml
ubuntu@ubuntu-server:~/config-test$ cat config1.yml
port: 8443
# port: 8080
transport: tcp
```

Pour tous les fichiers listés dans **`/tmp/liste.txt`** et remet leur version sauvegardée .bak à la place des fichiers actuels.

Q3.5 : La bonne procédure à suivre est la suivante :
– Vérifier le résultat de la modif à blanc.
– Appliquer la modification avec -i.bak sur un fichier
– Appliquer la modification avec -i.bak sur tous les fichiers.
– Restaurer les fichiers si un problème survient avec une boucle for et mv du .bak pour l'appliquer sur le fichier principal.

3.3 grep et expressions régulières sur les journaux

```
ubuntu@ubuntu-server:~$ grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' journal.txt | sort -u
10.0.0.5
172.16.0.3
192.168.1.10
192.168.1.20
ubuntu@ubuntu-server:~$ cat journal.txt
2026-10-08 09:00 INFO Connexion depuis 192.168.1.10
2026-10-08 09:05 ERROR Echec depuis 192.168.1.20
2026-10-08 09:05 ERROR Contact admin@entreprise.local
2026-10-08 09:15 INFO Contact client@gmail.com
2026-10-08 10:00 ERROR Echec depuis 10.0.0.5
2026-10-08 10:00 CRITICAL panic detecte
2026-10-08 10:15 INFO Connexion depuis 172.16.0.3
2026-10-08 11:00 ERROR Contact support@entreprise.local
2026-10-08 11:00 INFO Contact test@yahoo.com
2026-10-08 11:30 INFO Connexion depuis 192.168.1.10
```

Création du fichier journal.txt, et lancement de la commande :
grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' journal.txt | sort -u
`sort -u` trie les résultats et supprime les doublons.
[0-9]{1,3} signifie que ça peut être un nombre entre 0 et 9, trois fois de suite. exp: 192

En clair, ceci extrait les adresses composées de x.x.x.x.

```
ubuntu@ubuntu-server:~$ grep -oE '[a-zA-Z0-9._%+-]+@entreprise\.local' journal.txt | sort -u
admin@entreprise.local
support@entreprise.local
ubuntu@ubuntu-server:~$ grep -oE '[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}' journal.txt | grep -v '@entreprise\.local' | sort -u
client@gmail.com
test@yahoo.com
```

Il extrait les adresses de messageries intérieure puis extérieure.

```
ubuntu@ubuntu-server:~$ grep "ERROR" journal.txt | grep -oE '^[0-9-]+ [0-9]{2}' | uniq -c
      2 2026-10-08 09
      1 2026-10-08 10
      1 2026-10-08 11
```

Cela me permet d'extraire les erreurs et de les trier par ordre chronologique grâce à uniq -c

```
ubuntu@ubuntu-server:~$ grep -n -C3 "CRITICAL" journal.txt | head -40
3-2026-10-08 09:05 ERROR Contact admin@entreprise.local
4-2026-10-08 09:15 INFO Contact client@gmail.com
5-2026-10-08 10:00 ERROR Echec depuis 10.0.0.5
6:2026-10-08 10:00 CRITICAL panic detecte
7-2026-10-08 10:15 INFO Connexion depuis 172.16.0.3
8-2026-10-08 11:00 ERROR Contact support@entreprise.local
9-2026-10-08 11:00 INFO Contact test@yahoo.com
```

Avec -C, ça affiche trois lignes avant et après pour l'état Critical.

```
ubuntu@ubuntu-server:~$ if grep -q "panic" journal.txt; then echo "alerte" ; else echo "aucun incident" ; fi
alerte
ubuntu@ubuntu-server:~$
```

Si en cherchant dans le fichier journal.txt il trouve le mot "Panic", il écrit alerte, sinon aucun incident.

```
ubuntu@ubuntu-server:~$ grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b' journal.txt | sort -u | wc -l
4
```

Point de contrôle 3.3, wc est un compteur de lignes de caractères ou de mots et -l précise l'option "lignes".
Ma sortie est de 4 lignes pour des adresses distinctes.

```
ubuntu@ubuntu-server:~$ if grep -q "motif-inexistant" journal.txt; then echo "alerte" ; else echo "aucun incident" ; fi
aucun incident
ubuntu@ubuntu-server:~$
```

Cette fois j'ai recherché une chaîne de caractères qui n'existe pas avec un si, sinon.

Q3.7 — Quelle différence entre grep 'abc' , grep '^abc' et grep 'abc$' ?
Le premier affiche recherche n'importe où dans un fichier, le deuxième cherche à partir du début du fichier, le dernier à partir de la fin.

Q3.8 — Pourquoi le tri par heure donne-t-il un ordre faux sans précaution ?
Selon mes recherches, le tri par heure compare les caractères et non les valeurs numériques en elles-mêmes.

Q3.9 — Comment vérifier dans un script qu'un motif critique est absent d'un journal ?
Pour vérifier la commande comme utilisé précédemment, $grep -q avec des conditions if et else et echo pour afficher un résultat sur le terminal.

3.4 Pipelines et oneliners d'exploitation

```
ubuntu@ubuntu-server:~$ tail -n 1000 access.log | awk '{print $1}' | sort | uniq -c | sort -rn | head -10
      6 192.168.1.30
      4 192.168.1.20
      4 192.168.1.10
      3 192.168.1.50
      3 192.168.1.40
      3 10.0.0.8
      2 192.168.1.70
      2 192.168.1.60
      2 10.0.0.5
      1 172.16.0.3
```

Les adresses triées par fréquence avec le nombre de fois d'apparition.

```
ubuntu@ubuntu-server:~$ df -P | awk 'NR>1 && $5+0 > 10 {printf "%-24s %5s\n", $6, $5}'
/                        11%
/boot                    12%
ubuntu@ubuntu-server:~$
```

Le concept de one-liner va me permettre de combiner plusieurs commandes en une, séparée par “|”. En l’occurrence, il affiche les volumes ayant plus de 10 % de leur stockage utilisé.

```
ubuntu@ubuntu-server:~$ sudo du -ah /var | sort -rh | head -10
[sudo] password for ubuntu:
504M    /var
225M    /var/lib
201M    /var/lib/apt/lists
201M    /var/lib/apt
144M    /var/log
141M    /var/log/journal/ce353e2d51084c82ac2c8c45a4067c9c
141M    /var/log/journal
135M    /var/cache
112M    /var/cache/apt
70M     /var/lib/apt/lists/archive.ubuntu.com_ubuntu_dists_noble_universe_binary-amd64_Packages
```

df -P et awk listent les dix plus gros fichiers de /var.

```
ubuntu@ubuntu-server:~$ ss -tulpn | awk 'NR>1 {print $1, $5}' | sort -u
tcp 0.0.0.0:22
tcp 127.0.0.53%lo:53
tcp 127.0.0.54:53
tcp [::]:22
udp 127.0.0.53%lo:53
udp 127.0.0.54:53
```

Le one-liner liste les ports en écoute.
– tulpn affiche le protocole TCP, UDP, affiche les processus et affiche les adresses et ports.

```
ubuntu@ubuntu-server:~$ mkdir -p ~/test-remplacement
ubuntu@ubuntu-server:~$ printf 'ancien\ntexte ancien\nrien ici\n' > ~/test-remplacement/exemple.txt
ubuntu@ubuntu-server:~$ grep -rl 'ancien' ~/test-remplacement

ubuntu@ubuntu-server:~$ grep -rl 'ancien' ~/test-remplacement | xargs -r sed -i.bak 's/ancien/nouveau/g'
ubuntu@ubuntu-server:~$ cat ~/test-remplacement/exemple.txt
nouveau
texte nouveau
rien ici
ubuntu@ubuntu-server:~$ cat ~/test-remplacement/exemple.txt.bak
ancien
texte ancien
rien ici
```

Là je m’inspire des commandes précédentes pour rechercher une chaine de caractères dans un répertoire, en créant d'abord un répertoire sur la racine et en écrivant trois lignes dans le nouveau fichier.

```
ubuntu@ubuntu-server:~$ cat ~/test-remplacement/exemple.txt.bak
ancien
texte ancien
rien ici
```

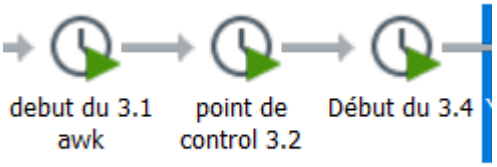
La sauvegarde a bien été faite.

Q3.10 — Décomposez le pipeline des dix adresses les plus actives : que fait chaque maillon
Il met les différentes adresses IPv4 dans l'ordre décroissant des adresses les plus actives.

Q3.11 — À quoi sert xargs dans le dernier oneliner ? Que se passerait-il sans lui ?

`xargs` sert à transmettre à une commande les noms de fichiers trouvés par la commande précédente. Sans cela, on ne pourrait pas remplacer les valeurs.

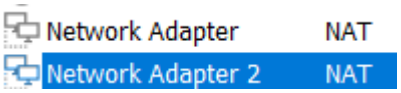
Q3.12 — Pourquoi le tri par fréquence doit-il être numérique et inverse ?
Pour comparer correctement les fréquences numériques.



Les snapshots pour le TP 3.x

TP 4.1 : Deux interfaces réseau : agrégation et bascule

4.1.1
Ajouter une seconde carte réseau à la machine virtuelle



4.1.2

```
[badis@rhel9-lab ~]$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:22:f0:ff brd ff:ff:ff:ff:ff:ff
    altname enp3s0
3: ens224: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:22:f0:09 brd ff:ff:ff:ff:ff:ff
    altname enp19s0
```

```
[badis@rhel9-lab ~]$ nmcli con show
NAME                UUID                                  TYPE      DEVICE
ens160              5685424c-ac9a-30be-b46e-85e04e2b2bc8  ethernet  ens160
Connexion filaire 1 78faae05-a395-3c04-86f6-131736d53c26  ethernet  ens224
bond0               183ace69-19c9-4c32-a76e-5ae3efc56c8a  bond      bond0
lo                  b7be2ef5-ed6c-4233-9523-c294c381a30c  loopback  lo
bond0-port1         1db2f15f-80fb-4ed4-981a-54b8af3cc505  ethernet  --
bond0-port2         7cc51587-afba-4279-9916-192e08838632  ethernet  --
```

bond0-port2 et bond0-port1 = ethernet
Pour vérifier que le bond : nmcli con show

sudo nmcli con add type bond con-name bond0 ifname bond0 bond.options "mode=active-backup,miimon=100"

bond = crée une agrégation de carte
mode=active-backup : c'est le mode actif/secours

4.1.3
On rattache les deux interfaces à l'agrégation.
on doit attribuer une adresse fixe du bond : 10.0.0.10

```
[badis@rhel9-lab ~]$ ip -br a
lo               UNKNOWN        127.0.0.1/8 ::1/128
ens160           UP
ens224           UP
bond0            UP             10.0.0.10/24 fe80::ced1:4fb8:e5c3:b5e1/64
```

```
[badis@rhel9-lab ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc mq master bond0 state UP group default qlen 1000
    link/ether 00:0c:29:22:f0:ff brd ff:ff:ff:ff:ff:ff
    altname enp3s0
3: ens224: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc mq master bond0 state UP group default qlen 1000
    link/ether 00:0c:29:22:f0:ff brd ff:ff:ff:ff:ff:ff permaddr 00:0c:29:22:f0:09
    altname enp19s0
4: bond0: <BROADCAST,MULTICAST,MASTER,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:22:f0:ff brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.10/24 brd 10.0.0.255 scope global noprefixroute bond0
       valid_lft forever preferred_lft forever
    inet6 fe80::ced1:4fb8:e5c3:b5e1/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Q4.1 — Quelle différence entre le mode actif/secours et l'agrégation de liens ? Dans quel cas chacun convient-il ?

En actif/secours, la carte réseau travail et quand elle tombe, l'autre prend le relais. C'est pour la fiabilité. L'agrégation utilise toutes les cartes pour plus de débit.

Q4.2 — Pourquoi l'adresse doit-elle être portée par l'agrégation et non par une interface membre ?

Si l'adresse est sur une carte et qu' elle tombe alors la machine n'est plus joignable.
L'adresse ip reste la même sur le bond0

Q4.3 — Comment prouver réellement la continuité de service pendant une bascule ?

On lance un ping continu et on coupe la carte active avec nmcli device disconnect, pour déterminer la continuité ou les pertes.